

cannot be completed without first determining whether all containers operating on the platform are compatible with newer versions. To ensure that all apps function properly, the product team must meticulously organise upgrade tasks. When we employ open source components that are backed/managed by a third party organisation, the support life can be prolonged, as is the case when we use AWS's managed Kubernetes service (Elastic Kubernetes Service).

Upgrades required due to new features

Sometimes we need to upgrade open source components since new features are launched in the projects and we want to integrate them into our product design. This includes optimisation, which is typically associated with improved performance.

Because open source products/components are utilised by a wide range of consumers, most new improvements offer value and address long-standing concerns reported by open source project participants.

There is an excellent blog post at <https://layer5.io/blog/kubernetes/kubernetes-129-highlights-features-and-deprecations> that discusses the many features that will be included in Kubernetes version 1.29. Assuming your project intends to use the Gateway API, you'll want to update to the most recent Kubernetes version. Refer to <https://gateway-api.sigs.k8s.io/> for further information about the Gateway API.

Upgrades required due to software bugs

Open source components are utilised by a diverse range of community members and become more stable as the user base grows. However, it is probable that you will encounter problems in the open source software, causing production troubles. If you're lucky, the same issues will be handled

in subsequent versions of components, and you'll need to upgrade the component to address production difficulties. The speed with which you wish to upgrade the component to a newer release is entirely dependent on the sort of issue and its frequency.

Upgrades required due to security bugs

Security flaws represent substantial threats to systems and data, potentially resulting in unauthorised access, data breaches, or system compromise. Patching security issues as soon as possible helps to reduce vulnerabilities and improves the overall security posture of the program or system. Failure to fix security flaws exposes systems to exploitation by unscrupulous actors, potentially resulting in financial losses and reputational harm. Patching security flaws on a regular basis is critical for ensuring system and data integrity, confidentiality, and availability.

Security is an important feature of any production quality program, and if a problem is reported in an open source component used in the product, its reason enough to upgrade the software. Snyk is open source software that automates vulnerability repairs with a one-click pull request containing the necessary updates and patches.

Well planned upgrades

Ideally, every project and module owner should plan for frequent component updates without waiting for any of the reasons listed above. Because each upgrade necessitates a significant amount of product testing,

these events must be carefully planned. Unplanned incidents may not receive adequate testing coverage. There are certain tools that might help you plan your module updates.

Dependabot is a program that automates the process of updating software dependencies. It scans your repository for obsolete dependencies and creates pull requests to update them to the newest version. This is an excellent indicator for product owners as to what fresh modifications are being made in the open source components.

Based on the alerts received from Dependabot or any other utility, thorough testing can be done to rollout product changes.

Any moderate-sized project has 100 or more open source dependencies. Monitoring modifications to open source components is crucial for several reasons. For starters, it might result in the discontinuation of updates, including security fixes, leaving systems open to vulnerabilities. Second, it indicates the end of official support, which may result in compatibility concerns or reliance on old technology. Third, without regular maintenance, projects may become incompatible with changing software environments, impeding development. Furthermore, ageing components may lack community participation, which reduces the availability of resources, documentation, and technical support. Finally, remaining updated about open source components aids in migration or replacement planning, as well as assuring software system life span and security. 

Reference

- <https://layer5.io/blog/kubernetes/kubernetes-129-highlights-features-and-deprecations>

By: Ajay Lotan Thakur

The author is a senior IEEE member, BCS fellow, TST member of ONF's open source Aether (Private 5G) Project, and cloud software architect at Intel, Canada.